

MiniTorch: A Modular Educational Tensor and Automatic Differentiation Framework

Naga Sampath Maddineni
University of North Texas

Abstract— MiniTorch is a from-scratch educational deep-learning framework developed as a modular Python project. The implementation builds tensor operations, automatic differentiation, module abstractions, and verification workflows without relying on a full production framework for the core mechanics. This report documents the repository evidence, development methodology, testing strategy, and scope limitations of the implementation.

Index Terms— automatic differentiation, tensors, deep learning systems, numerical programming, test-driven development.

I. INTRODUCTION

MiniTorch provides a compact environment for studying the mechanisms behind modern deep-learning libraries. The local project contains multiple course modules, setup metadata, implementation packages, project scripts, and tests. The public portfolio version preserves the implementation and reproducibility guidance while excluding local virtual environments and generated caches.

II. METHODOLOGY

The work was organized incrementally. The first module establishes the core numerical and tensor abstractions, while later module material extends the framework toward differentiable computation and higher-level project behavior. Python package metadata, setup files, requirements files, and test modules provide the executable structure. Pytest and Hypothesis are used to exercise expected behavior and numerical properties; NumPy supplies the array representation used by the educational implementation.

III. EXPERIMENTS AND RESULTS

The repository evidence includes module-specific tests, project scripts, setup configuration, and a visualization artifact. These artifacts demonstrate that the project was evaluated through executable checks rather than only narrative documentation. Because the original submission does not provide a single consolidated benchmark table, this report intentionally does not claim aggregate accuracy, runtime, or coverage metrics. Reproduction should begin with the module requirements and test commands documented in the repository.

IV. LIMITATIONS

MiniTorch is an educational framework and is not intended to replace optimized production libraries. The public bundle does not include local virtual environments, generated Hypothesis examples, caches, or model checkpoints. Results may depend on Python, NumPy, Pytest, and Hypothesis versions; a future revision should pin versions and add a consolidated continuous-integration summary.

V. CONCLUSION

MiniTorch demonstrates the design of a small, testable deep-learning substrate using Python and NumPy. Its modular organization makes the implementation suitable for studying tensors, automatic differentiation, and framework abstractions, while the public repository provides a clean boundary between reproducible source artifacts and local/generated files.

REFERENCES

- [1] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [2] pytest documentation, “The pytest framework,” Python Software Foundation, accessed 2026.
- [3] Hypothesis documentation, “Property-based testing for Python,” accessed 2026.